

Desarrollo

- 1. Client-Server Architecture** El dispositivo móvil funciona como cliente que envía solicitudes a servidores remotos mediante HTTP o WebSocket. Librerías como Retrofit o Volley gestionan esa capa de transporte.
- 2. IP Address** Identifica de forma única a cada dispositivo en una red. Al usar Retrofit, el sistema operativo resuelve la IP automáticamente, por lo que el desarrollador trabaja solo con URLs legibles.
- 3. DNS** Traduce un nombre de dominio en la IP real del servidor. OkHttp realiza esta resolución en segundo plano cada vez que la app llama a un endpoint.
- 4. Proxy / Reverse Proxy** El proxy de cliente permite depurar tráfico con herramientas como Charles Proxy. El reverse proxy opera en el servidor balanceando solicitudes, y la app lo consume sin distinguirlo de un servidor normal.
- 5. Latency** Tiempo entre que se envía una solicitud y se recibe la respuesta. Las llamadas de red deben ejecutarse en hilos secundarios con Coroutines o RxJava para no bloquear la interfaz.
- 6. HTTP/HTTPS** Desde la API 28, Android bloquea el tráfico sin cifrar por defecto. Toda comunicación debe ir sobre HTTPS, declarando los permisos correctos en el manifiesto.
- 7. APIs** Interfaz que conecta la app con el backend. Retrofit es la librería más adoptada en Android porque permite definir llamadas de red de forma declarativa.
- 8. REST API** Arquitectura basada en verbos HTTP como GET, POST o DELETE sobre recursos definidos. Es el enfoque más extendido para consumir datos en apps móviles.
- 9. GraphQL** Permite al cliente solicitar solo los campos necesarios, evitando exceso o falta de datos. Ventajoso en mobile por el ancho de banda limitado; Apollo Client es la librería principal en Kotlin.
- 10. Databases** Las apps usan almacenamiento local con Room/SQLite y remoto con Firebase o Supabase. Los repositorios orquestan caché, sincronización y soporte offline.
- 11. SQL vs NoSQL** Room representa el modelo relacional para datos estructurados, mientras que Firestore o Realm ofrecen un enfoque no relacional para datos flexibles y sincronización en tiempo real.
- 12. Vertical Scaling** Agregar más recursos a un servidor. En mobile su analogía es optimizar el dispositivo: reducir consumo de memoria y evitar operaciones costosas en el hilo principal.
- 13. Horizontal Scaling** El backend escala agregando más instancias. Con escalado horizontal y CDN, el impacto sobre los servidores de origen se reduce considerablemente.
- 14. Load Balancers** Distribuyen el tráfico entre múltiples servidores. La app llama al mismo endpoint y el balanceador decide internamente qué servidor atiende cada solicitud.
- 15. Indexing** Acelera búsquedas en base de datos. En Room, un índice sobre una columna frecuente puede convertir una búsqueda de miles de filas en una operación casi instantánea.
- 16. Replication** Firebase y Firestore mantienen réplicas con soporte offline. Al recuperar conectividad, los cambios locales se fusionan con el servidor sin pérdida de datos.

17. Sharding Distribuye datos entre múltiples bases según región o ID de usuario. El backend puede implementarlo de forma completamente transparente para el desarrollador móvil.

18. Vertical Partitioning En Room se puede dividir una entidad en varias tablas con menos columnas, reduciendo el volumen leído en cada consulta y mejorando el rendimiento.

19. Caching OkHttp cachea respuestas HTTP, Glide o Coil cachean imágenes, y Room persiste datos estructurados. Estas capas reducen la latencia y permiten funcionamiento parcial sin conexión.

20. Denormalization Almacena datos redundantes en Room para evitar joins costosos, permitiendo obtener datos en una sola consulta sin operaciones adicionales.

21. CAP Theorem Un sistema distribuido no puede garantizar simultáneamente consistencia, disponibilidad y tolerancia a particiones. Firestore prioriza disponibilidad y tolerancia, ofreciendo consistencia eventual.

22. Blob Storage Imágenes y archivos pesados se almacenan en Firebase Storage o AWS S3, accedidos mediante URLs firmadas. El streaming de video usa protocolos como HLS o DASH.

23. CDN Red de servidores distribuidos que entrega contenido estático desde el nodo más cercano al usuario. Glide o Picasso se benefician automáticamente cuando las URLs apuntan a un CDN.

24. WebSockets Conexión persistente y bidireccional para chat en tiempo real, juegos o notificaciones instantáneas. OkHttp incluye soporte nativo y Scarlet integra Coroutines.

25. Webhooks Como los dispositivos no exponen un endpoint público, los webhooks del servidor se traducen en push notifications vía Firebase Cloud Messaging.

26. Microservices Divide el backend en servicios independientes. La app se comunica con ellos directamente o a través de un API Gateway que unifica los endpoints.

27. Message Queues La app recibe respuesta inmediata mientras el servidor encola tareas en RabbitMQ o Kafka para procesarlas de forma asíncrona, evitando esperas por operaciones costosas.

28. Rate Limiting El servidor limita peticiones por usuario o IP. En el cliente se implementa backoff exponencial para manejar rechazos sin saturar aún más el servidor.

29. API Gateways Punto de entrada único que enruta llamadas hacia los microservicios. Centraliza autenticación, rate limiting y logging, simplificando la lógica en el cliente.

30. Idempotency Garantiza que ejecutar la misma operación múltiples veces produce el mismo resultado, evitando que reintentos automáticos dupliquen acciones como cobros o pedidos.